Hack Machine Language / Hack Assembly

1. Introduction

- Hack machine language is a simple, binary-based language created for the Hack computer in the Nand2Tetris course.
- Its design is minimal, with just two kinds of instructions, making it ideal for learning how computers process instructions from hardware to software.

2. Structure of Hack Machine Language

- All Hack machine instructions are 16 bits long.
- The language supports only two instruction formats:
    - A-instruction: Used for memory addressing.
    - C-instruction: Used for computations and program control.

3. A-Instruction

Format

- Begins with a '0'
- Structure: 0vvvvvvvvvvvvvvv
- The remaining 15 bits represent either a value or a memory address.

Role

- Used to set the A Register to a value (for direct addressing) or point to a memory location (for indirect addressing).
- Examples:
    - @21 (sets A register to decimal 21)
    - @LOOP (sets A register to the address labelled 'LOOP')

4. C-Instruction

Format

- Begins with '111'

- Structure: 111accccccdddjjj
    - a: Determines if computation uses memory (M) or register (A)
    - comp (cccccc): Specifies the computation (like D+1, M-1, D|A, etc.)
    - dest (ddd): Destination (A, D, or M) where result is stored
    - jump (jjj): Used for branching/jumping (JMP, JGT, JEQ, etc.)

Role

- Performs computations: arithmetic, logic, data storage.
- Handles conditional and unconditional program jumps.

Examples

- D=M (copy memory value at address stored in A to D register)
- D=D+A (add D and A, store in D)
- 0;JMP (jump to address in A)

The D, A, and M Meaning

- D: Data Register — used to store intermediate results, computation values.
- A: Address Register — stores the address for memory access or can be used for math.
- M: Memory — refers to the memory location pointed to by the value in A register.

Usage:

- @15 → Put 15 in A register.
- M=5 → Store 5 at memory location in A register.
- D=M → Copy memory (RAM) at A register's address into D register.

5. Instruction Sets in Hack Machine Language

- Data Movement: D=A, M=D, D=M
- Arithmetic Operations: D=D+M, D=D+1, A=A-1, etc.
- Logic Operations: D=D&A, D=D|M, !D
- Branching/Jump: D;JGT, D;JEQ, D;JLT, 0;JMP
- Memory Access: Two-step - set address with @address, then use M to work on memory

6. Symbols and Labels

- Symbols (labels) allow reference by name instead of address.
- Assembly program: symbol table records where each named label is (e.g., LOOP).

7. Hack Programming: Assembly Language

- Hack assembly is a symbolic, human-readable layer above machine code. You write instructions as mnemonics.
- Examples:
  @2
  D=A
  @3
  D=D+A
  @0
  M

  Typical Process:
  - Use @ to refer to memory addresses.
  - Perform calculations and store results.

8. Writing and Running Hack Programs

Stepwise Approach:

1. Write assembly code (.asm) using A & C instructions, labels, comments
2. Convert to machine language (.hack) using an assembler
3. Load machine code into the Hack computer (hardware, simulator, or VM)
4. Run and observe output or effects on memory/registers

9. Programming Examples

(A) Adding Two Numbers:

```
// Program to add 2 + 3 and store result at RAM[0]
@2
D=A
@3
D=D+A
@0
M=D
```

//Sets D to 2, adds 3, stores at RAM.

(B) Infinite Loop Example:

```
(LOOP)
@LOOP
0;JMP
```

//Repeats forever

10. Programming Best Practices

- Use labels for code organization.
- Add comments for clarity: // This sets D to A
- Sequence: address → compute → store or jump

11. Debugging and Simulation

- Use Hack Simulator to load and step through code.
- Track register values and RAM space.

12. Summary Table of C-Instruction Fields

Comp	Meaning

0	Literal 0
1	Literal 1
D	Register D
A	Register A
M	Memory
D+1	D plus 1
D+A	D plus A
D	A

13. Why Hack Programming Matters

- Teaches direct connection between code and hardware.
- Build foundational skills for understanding conventional CPUs and assembly languages.
- Gives a hands-on perspective for how high-level code (Jack, C, Python) is ultimately implemented.